LINQ

NET 022 - Advanced C# Programming

Prepared By: Supun Kandaudahewa
supun.kandaudahewa@humber.ca

HUMBER

WE ARE HUMBER

Agenda

- Traditional Querying
- What is LINQ
- Advantages of using LINQ
- LINQ Syntax
  - Query Syntax
  - Method Syntax
- Deffered Execution
- Standard Query Operations
- Exercises on selected/more frequently used methods
- More LINQ examples

Traditional Querying

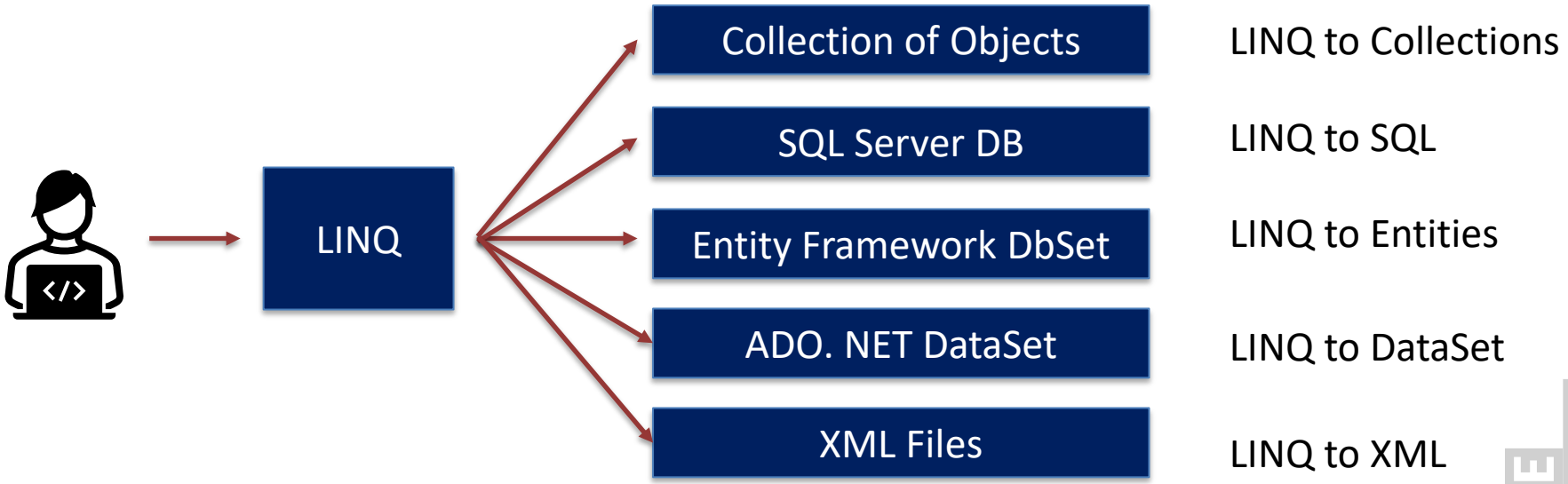
- Queries against the data are expressed as simple strings.

Standard Query
SELECT * FROM Customers WHERE FirstName LIKE 'K%'

- Downsides of tradition querying →
  - No type checking at compile time.
  - No IntelliSense support.
  - Query syntax is language specific.
    - SQL databases
    - Various web services
    - Reading xml documents

LINQ

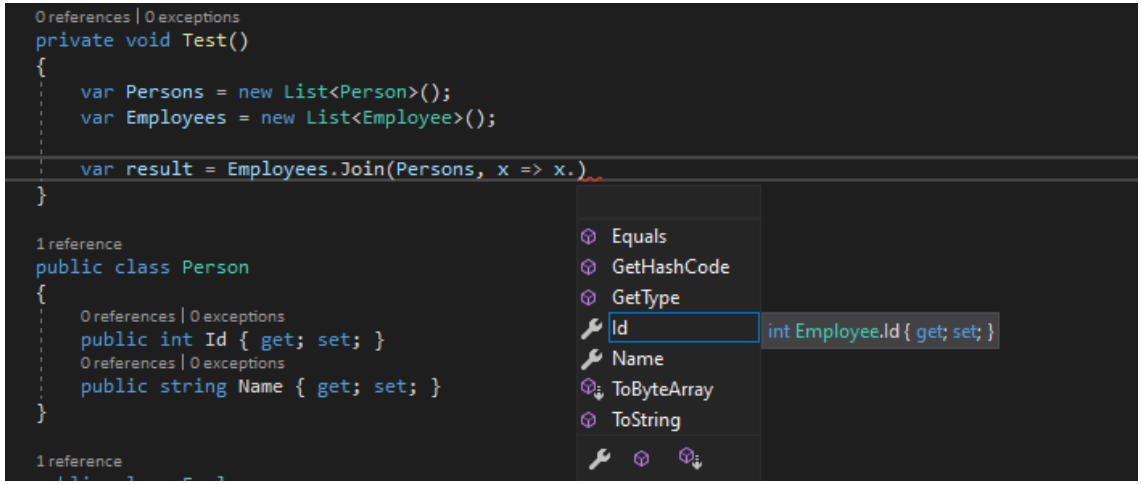
- **L**anguage-**I**ntegrated **Q**uery (LINQ) is the name for a set of technologies based on the **integration of query capabilities directly into the C# language**.
- It allows you to retrieve/query data from various data sources such as arrays, collections, databases, XML files etc.



Language Integrated Query (LINQ) in C# | Microsoft Learn

Advantages of LINQ

- Ability to query multiple data sources using SINGLE- syntax.
- Compile-Time type checking helps identifying query errors easily and at a very preliminary stage.
- IntelliSense Support.



Source: https://stackoverflow.com/

LINQ Syntax – Query Syntax

- Query syntax is the more declarative approach out of the two approaches.
- Bulkier when compared with method syntax.
- Preferred in complex situations, specially JOINS.
- Let's see how we can utilize query syntax to query data in an array.

https://dotnetfiddle.net/pO2UWY

Query Syntax

```
IEnumerable<int> query =
                    from number in numbers where number < 10
                    select number;
foreach(var number in query)
{
   Console.WriteLine(number);
}
```

LINQ Syntax – Method Syntax

- Produces more cleaner and concise code.
- In the .NET Framework (System.Linq).
- Supports method chaining. → this is extremely useful.
- Preferred in most of the situations.
- Let's see how we can utilize method syntax to query data in an array.

https://dotnetfiddle.net/x7deC9

Query Syntax

```
IEnumerable<int> query = numbers.Where(n=>n<10);;

foreach(var number in query)
{
   Console.WriteLine(number);
}
```

Deffered Execution

- LINQ operations are performed when the data is requested not when it is declared.
- It means that the evaluation of an expression is delayed until its value is required.
- It greatly improves performance by avoiding unnecessary execution.

  - When query variable is iterated over →foreach loop
  - When a ToList(), ToArray() etc. is applied.
  - When methods returning a scalar value is executed. SUM(), MIN(), MAX()

Deffered Execution Contd.

- Coding Exercise.

Standard Query Operations - 1

- **Filtering**
  - Where, OfType
- **Sorting**
  - OrderBy, OrderByDescending, ThenBy, ThenByDescending, Reverse
- **Grouping**
  - GroupBy
- **Aggregation**
  - Average, Count, Max, Min, Sum
- **Join**
  - Inner, Group, Left, Cross
- **Project**
  - Select, SelectMany
- **Quantifiers**
  - All, Any, Contains

Standard Query Operations - 2

- **Elements**
  - ElementAt, ElementAtOrDefault,First, Last, Single, FirstOrDefault, LastOrDefault, SingleOrDefault
- **Concatenation**
  - Concat
- **Partitioning**
  - Skip, Take, SkipWhile(), TakeWhile()
- **Conversion**
  - AsEnumerable, AsQueryable, Cast, ToArray, ToDictionary
- **Generation**
  - Empty, DefaultEmpty, Range, Repeat
- **Equality**
  - SequenceEqual
- **Set**
  - Distinct, Except, Intersect, Union

Where()

- Where() method filters collection based on given lambda expression and returns a new collection with matching elements.

Where() Extension Method - Declaration
Where(Func<TSource, bool> predicate)

- As Where method accepts a Func of given signature we can create a named method and pass it to Where method as well.
- Through the evolution of .Net creating delegates has been abstracted:
  - Named Method
  - Anonymous Method
  - Lambda

OrderBy()

- OrderBy() method sorts collection based on given Lambda expression (property) and returns a new collection with sorted elements.

OrderBy() Extension Method - Declaration
OrderBy(Func<TSource, TKey> keySelector)

- OrderByDescending()
- ThenBy()
- ThenByDescending()
- Reverse()

First(), Last()

- First() method returns first element in the collection that matches with the given condition.
- Last() method returns last element in the collection that matches with the given condition.
- Both First() and Last() will throw an exception if no element is present in the collection which matches the criteria.
- Instead, you can use FirstOrDefault() and LastOrDefault() which returns null if no element matches the condition.

First() Extension Method - Declaration
First(Func<TSource, bool> predicate)

Last() Extension Method - Declaration
Last(Func<TSource, bool> predicate)

Single(), SingleOrDefault()

- Single() method returns first element (only one) in the collection that matches with the given condition.
- If multiple matches are found or no match is found, it will throw an exception.
- SingleOrDefault() will return null if no matching elements are found but will throw an exception if multiple matches are found.

Single() Extension Method - Declaration
Single(Func<TSource, bool> predicate)

SingleOrDefault() Extension Method - Declaration
SingleOrDefault(Func<TSource, bool> predicate)

All(), Any()

- **All()** method is used to check whether all the elements of a data source satisfy a given condition or not. If all the elements satisfy the condition, then it returns true else return false.
- **Any()** is used to check whether at least one of the elements of a data source satisfies a given condition or not. If any of the elements satisfy the given condition, then it returns true else return false.
- It is also used to check whether a collection contains some data or not (Overload version)

Select()

- Both these methods are used for projection. Projection is nothing but the mechanism which is used to select the data from a data source.
  - In its original form
  - In a new form
- This is heavily used in object mapping. (Mapping one object to another type)

More examples

- Please use the below link to access more samples

try-samples/101-linq-samples at main · dotnet/try-samples · GitHub

THANK YOU.

HUMBER

WE ARE HUMBER